RETAIL DATA AGENT · PERSONAL DATA · AS OF 2026-11-01

BUILT 2026-11-01 · TICKET 10 · PRIVACY NOTE ADDED 2026-11-01

How do we stop a total from describing one person?

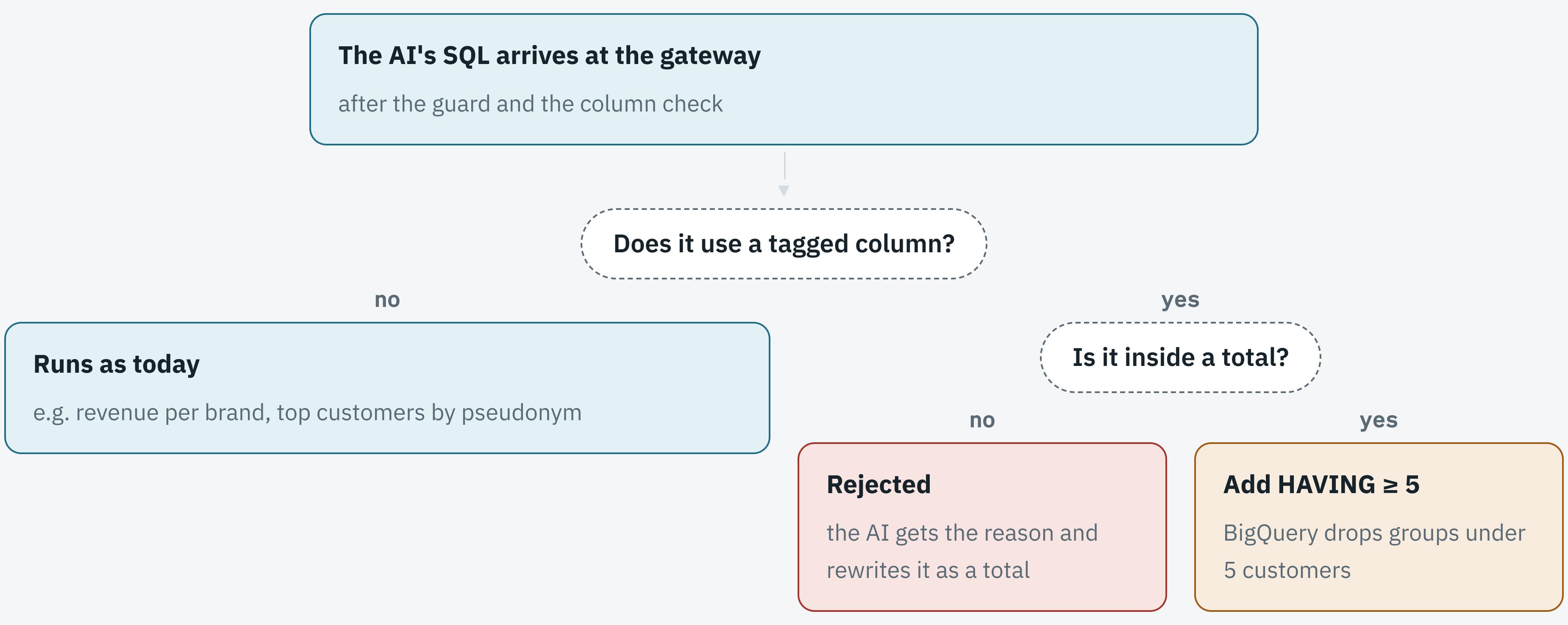
Columns like age and city are fine in a total over many customers, but a "total" over one customer is really that customer. We tag those columns in `schema.yaml`. The gateway then allows them only inside totals, and makes BigQuery drop every group with fewer than 5 customers. See also one query's full journey and every parser rule.

🟪 config: the tag and the limit 🟧 the gateway changes the SQL 🟦 runs as today 🟩 shown 🟥 rejected or hidden

In short

1. **Tag the columns.** In `schema.yaml`, age, gender, city, state, country and traffic source get `aggregate_only: true`.
2. **Listed one row per person?** Rejected. The AI is told to use the column only inside a total.
3. **Inside a total?** Allowed, and the gateway adds `HAVING COUNT(DISTINCT user_id) >= 5`, so BigQuery drops the small groups itself.
4. **No tagged column?** Nothing changes. "Top customers by spend" still works through pseudonyms.
5. **Rows were hidden?** The result tells the AI so, with a note under the table. It then says "hidden for privacy", never "no data".

The picture: two questions per query



The AI's SQL arrives at the gateway

after the guard and the column check

Does it use a tagged column?

no

Runs as today

e.g. revenue per brand, top customers by pseudonym

yes

Is it inside a total?

no

Rejected

the AI gets the reason and rewrites it as a total

yes

Add HAVING ≥ 5

BigQuery drops groups under 5 customers

🏠 Like a census office: it publishes counts per town, never a list of residents, and leaves out any town with fewer than 5 people.

What "tagging" means

A tag is one extra label next to a column in `schema.yaml`. The data doesn't change; the label tells the gateway how to treat that column, like a "fragile" sticker on a box. To protect another column, you edit the file, not the code.

Today

```
users:
  age:    {type: INT64}
  city:   {type: STRING}
  state:  {type: STRING}
  status: …
```

With the tag

```
users:
  age:    {type: INT64,  aggregate_only: true}
  city:   {type: STRING, aggregate_only: true}
  state:  {type: STRING, aggregate_only: true}
  status: …  # no tag: works as today
```

Tagged: `users` age, gender, city, state, country, traffic_source, plus `orders.gender`, which is the same fact copied onto every order.

Example: "average age of Levi's buyers in each city"

1. The AI writes a total, which is the allowed kind:

```
SELECT u.city, AVG(u.age) AS avg_age
FROM users u JOIN order_items oi ON oi.user_id = u.id
GROUP BY u.city
```

2. The gateway sees the tagged `city` and `age` inside a total, so it adds one line:

```
SELECT u.city, AVG(u.age) AS avg_age
FROM users u JOIN order_items oi ON oi.user_id = u.id
GROUP BY u.city
HAVING COUNT(DISTINCT u.id) >= 5   # 5 = MIN_GROUP_SIZE in .env
```

3. BigQuery counts the customers in each city and leaves out the small ones before sending anything back:

CITY	CUSTOMERS	AVG AGE	SENT TO THE AI?
Shanghai	412	38	✅ yes
São Paulo	297	35	✅ yes
~~Tromsø~~	~~1~~	~~27~~	❌ dropped: that's one person's age

Numbers made up to show the idea. The customers column is shown here only to explain; the AI never sees it.

Three queries, three outcomes

Listing a tagged column

```
SELECT id, city, age
FROM users
```

❌ Rejected. One row per person would show each customer's city and age.

A total of a tagged column

```
SELECT country, SUM(…)
… GROUP BY country
```

✅ Runs with `HAVING ≥ 5` added. Countries with 1–4 customers are left out.

No tagged column

```
SELECT user_id, SUM(…)
… GROUP BY user_id
```

✅ Unchanged. Each row is one customer, but only as a pseudonym like `cust_72ac84091163`.

`ARRAY_AGG` and `STRING_AGG` are rejected in these totals: they return one row per group, but it still holds every customer's value. Why `HAVING` and not dropping rows in Python afterwards: SQL applies `HAVING` before `LIMIT`, so "top 3 countries" still returns 3 safe rows instead of 3 rows minus the hidden ones.

The AI is told when rows were hidden

The AI never sees the `HAVING`: the gateway adds it after the AI wrote the SQL. So an empty result looked like "nothing there". Asked for the top 5 brands in Essen, where each brand had fewer than 5 buyers, it ran 4 queries hunting for a spelling mistake and then answered "no sales", which was false. Now the gateway marks the result, and the text the AI reads ends with a note.

Before: what the AI read

```
The query returned no rows.
Columns: brand, revenue
```

❌ "There are no sales in Essen." 4 queries.

Now: what the AI reads

```
The query returned no rows.
Columns: brand, revenue
Note: groups with fewer than 5 customers
are hidden for privacy, so there may be
data that can't be shown at this level
of detail.
```

✅ "The detail for Essen is hidden for privacy." 1 query.

When some rows came back, the note says instead that they **may not add up to the full total**. "257 cities in Germany" next to "4,198 German customers" no longer reads as if the cities hold everyone. A prompt rule backs the note up: never call it "no data", suggest a broader grouping (the country instead of one city), and don't re-run the query.

🏠 Like a waiter who can't forget the allergy warning, because the kitchen sticks a label on the plate.

What it doesn't stop

Subtracting two totals

"Total spend, all countries" minus "total spend, all countries except Iceland" gives Iceland's one customer. Each query passes on its own. Real protection needs added noise (differential privacy), which is beyond this prototype, so the design doc names it as a known limit.

> ### The rule in one sentence
> A tagged column can reach the AI only inside a total, and only when that total covers at least 5 customers, counted and enforced by BigQuery itself.

Ticket: `tasks/10-demographics-only-in-totals-of-5.md`. Plan: step 3 in `docs/plan.md`. Tests: `tests/test_step2_small_groups.py`.

Code: `query_safety/step2_personal_data/small_groups.py` (`SmallGroups.apply`), called from `QueryGateway.run` after the column check and before pseudonyms. Tag read in `schema.py` (`Column.aggregate_only`) from `schema.yaml`. Setting: `MIN_GROUP_SIZE` (default 5), written into the SQL as a plain number: it isn't secret, and the log shows exactly what ran.

The note: when `SmallGroups.apply` changed the SQL, the gateway sets `QueryResult.small_groups_hidden_below` (the minimum, else `None`), and `format_result` in `result_format.py` appends the note, the same way as the "only the first N rows" note. Prompt rule in `prompts/system.md`.

Each SELECT is checked on its own: a total inside a `WITH` step gets the `HAVING` on that step; a step that only passes `city` along is rejected. The id counted is the one of the table the tagged column comes from (`users.id`, `orders.user_id`).

See also one query's full journey, SQL parser rules, brand scoping, architecture.